# DETAILED DESIGN DOCUMENT

## Phishing URL Detection Module

Flowcharts • UML Diagrams • Data Flow Diagrams

Student Name: JEEVITHA L SHETTY
SRN : R23EQ045

## 1. Purpose and scope

This document expands the high-level system design for the Phishing URL Detection module into execution-level visual models. It contains flowcharts, UML class and sequence diagrams, and Level 0 and Level 1 Data Flow Diagrams (DFDs). It also includes flowcharts for the stated aim and three objectives. The diagrams follow the same architecture as Task 2: offline dataset preparation/training/evaluation is separated from online URL prediction.

The current prototype described in the supplied roadmap is URL-only: character n-gram TF-IDF is the main lightweight baseline, with a small NumPy character-CNN and an ensemble evaluated as alternatives. Registered-domain-disjoint splits, recall-oriented thresholding, URL-padding experiments and lightweight explanations are included in the design. DNS, TLS, webpage signals, Transformers, true temporal evaluation and production deployment are future extensions, not current capabilities.

## 2. Aim and objectives

Aim: To specify and evaluate a smart machine-learning module for phishing website detection using evidence from the reviewed literature.

- Objective 1: Survey and compare existing blacklist, heuristic, traditional ML, NLP/domain-aware, deep-learning, and hybrid detection approaches.
- Objective 2: Identify research gaps relating to generalization, feature diversity, false positives and false negatives, real-time detection, explainability, adaptation, and dataset quality.
- Objective 3: Derive functional and non-functional requirements and define an evaluation plan for the proposed module.

Figure 2. Aim and objectives alignment flowchart

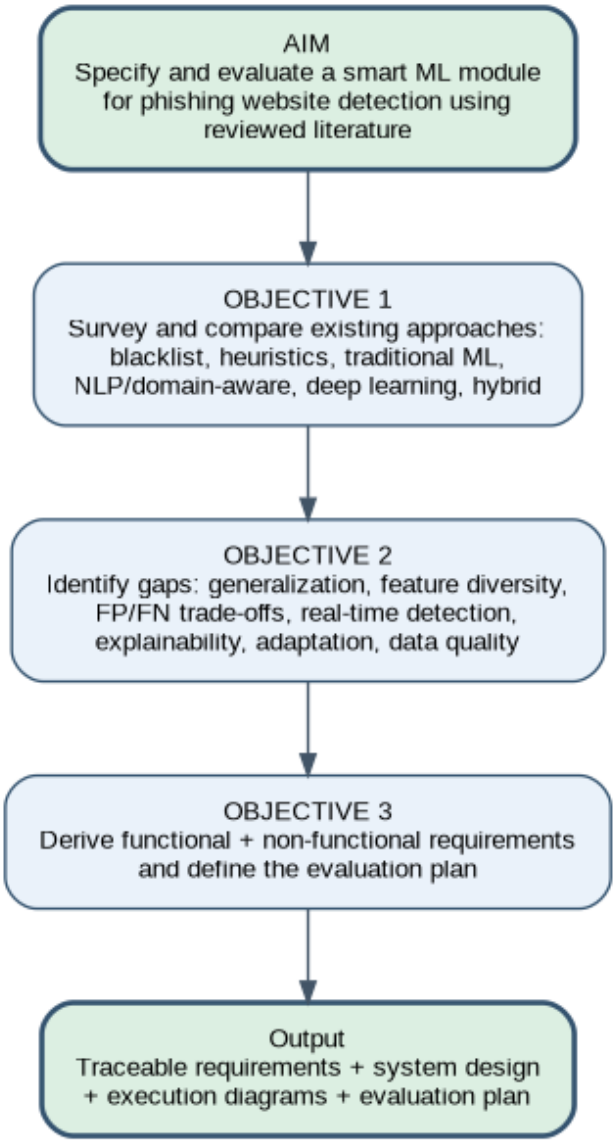



*Figure 1. Flowchart connecting the aim to Objectives 1–3 and the expected design outputs.*

Figure 3. Research-to-design workflow for Objectives 1–3




*Figure 2. Execution workflow for completing Objectives 1–3.*

## 3. Runtime flowchart

This flowchart shows the online execution path for one submitted URL. Invalid input or an incompatible model package is rejected safely. The prediction path does not automatically open or fetch the destination URL.

Figure 1. Runtime execution flowchart

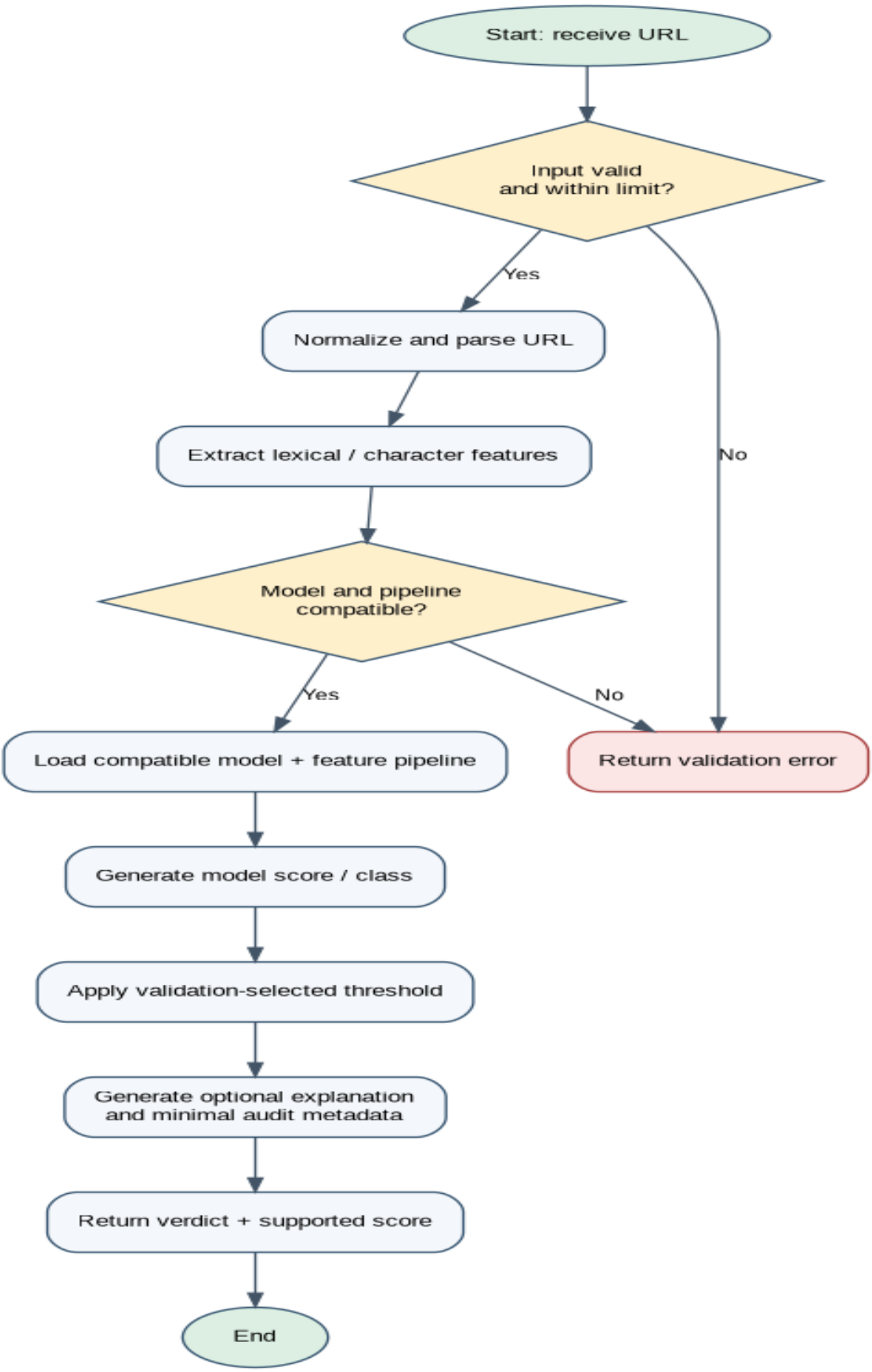


*Figure 3. Runtime decision flow for phishing URL detection.*

# 4. UML diagrams

## 4.1 UML class diagram

The class diagram shows the logical responsibilities and relationships between request handling, validation, feature extraction, model inference, threshold policy, explanation and response construction. It is a design-level model; names can be mapped to implementation classes or functions.

Figure 4. UML class diagram (logical design)

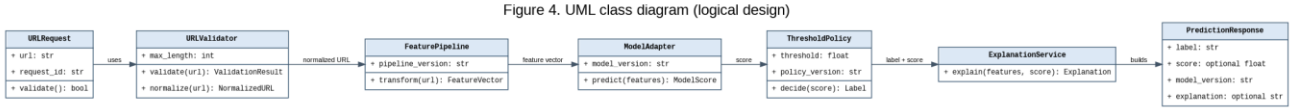



*Figure 4. UML class diagram showing the logical component relationships.*

## 4.2 UML sequence diagram

The sequence diagram models the messages exchanged for one prediction request. The dashed validation return represents the error branch; the green path represents the normal normalized-URL handoff.

Figure 5. UML sequence diagram — one prediction request

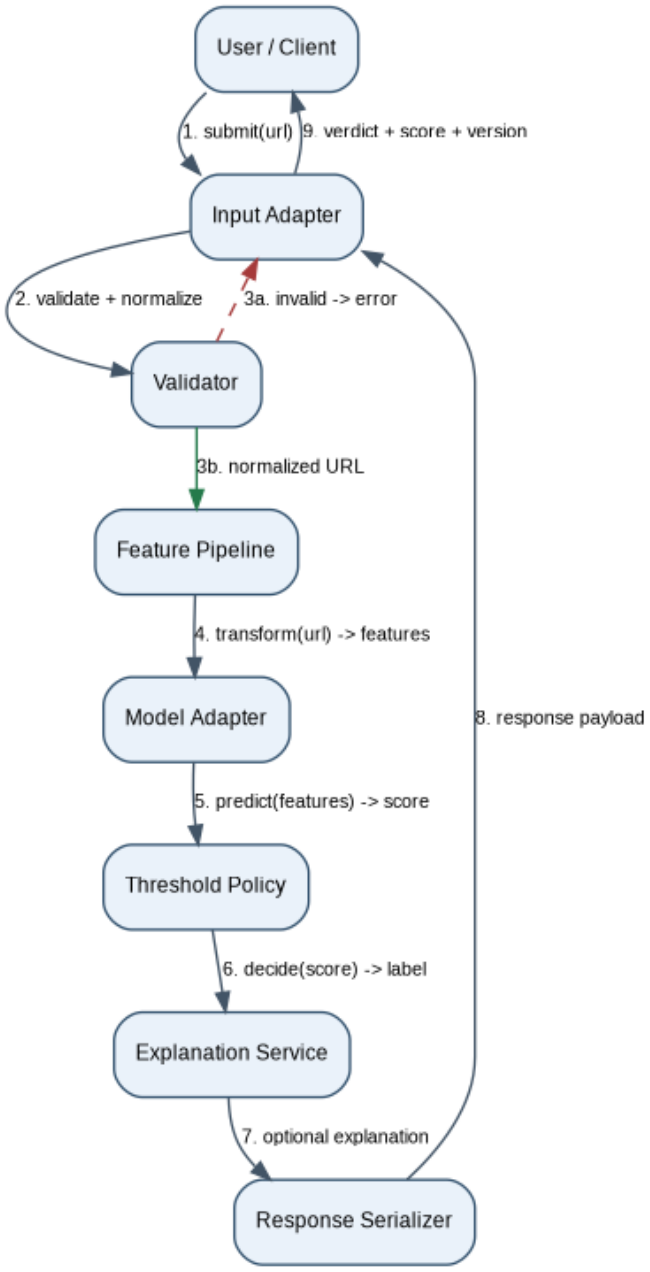



*Figure 5. UML sequence diagram for a single prediction request.*

## 5. Data Flow Diagrams (DFD)

DFD conventions used here: external entities are rectangles, processes are ellipses, and data stores are cylinder shapes. Arrows are labelled with the data being transferred. The offline training/evaluation pipeline is shown separately from online prediction to avoid mixing model-development data with live request data.

### 5.1 DFD Level 0 — Context diagram

The module is represented as one process. The user sends a URL and receives a verdict, supported score and optional explanation. The researcher/maintainer supplies labelled data or configuration and receives evaluation reports.

Figure 6. Data Flow Diagram — Level 0 (context)




*Figure 6. Level 0 DFD: system boundary and external data exchanges.*

### 5.2 DFD Level 1 — Decomposition

The Level 1 DFD decomposes the module into URL validation, feature extraction, inference/thresholding, response generation, and offline audit/training/evaluation. D1 stores the versioned model package, vectorizer and threshold; D2 stores the offline dataset and split manifest; D3 stores metrics, reports and model manifest.

Figure 7. Data Flow Diagram — Level 1

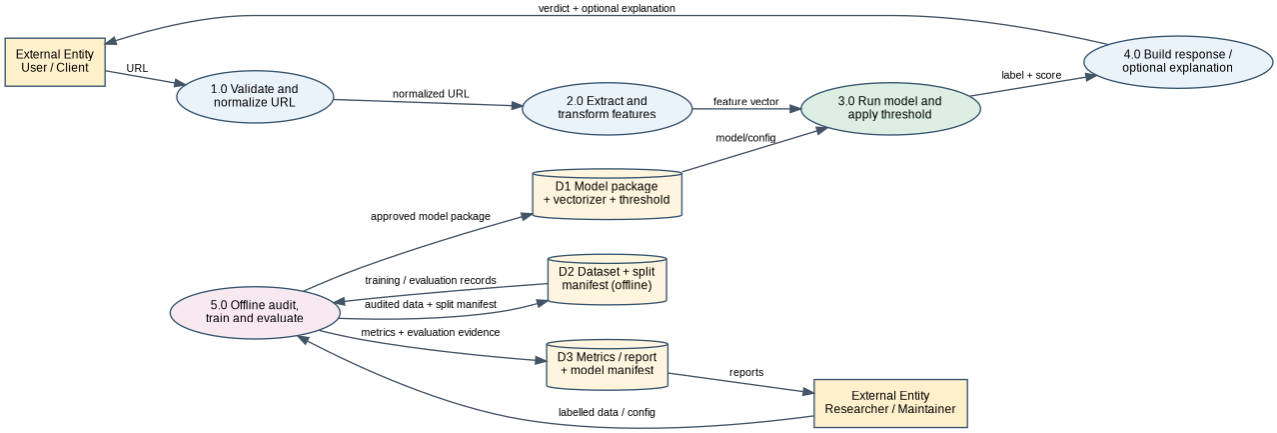



*Figure 7. Level 1 DFD: processes, stores and data movement.*

## 6. Module specifications

| ID / module | Inputs | Processing | Outputs |
|---|---|---|---|
| M1 — Input adapter | URL text from user/client | Check request shape and basic input limits | Validated request or input error |
| M2 — Validator / normalizer | Raw URL string | Validate syntax; apply documented normalization rules | Normalized URL and validation metadata |
| M3 — Dataset audit / split builder | Labelled URLs, sources and domain metadata | Check duplicates, missing values, source shortcuts and registered-domain overlap; split by domain | Audited data and saved split manifest |
| M4 — Feature pipeline | Normalized URL | Extract lexical/character features and apply the fitted vectorizer | Feature vector or sparse matrix |
| M5 — Model adapter | Features plus matching model package | Run TF-IDF classifier or selected optional CNN/ensemble | Model score or class |
| M6 — Threshold policy | Model score and frozen threshold config | Apply selected operating point | Phishing/Legitimate label |
| M7 — Explanation/audit | Features, model result and version metadata | Generate supported feature explanation; log minimal operational metadata | Optional explanation and audit event |
| M8 — Evaluation service | Candidate model, held-out splits and transformations | Compute metrics, domain-level results, latency and uncertainty estimates | Evaluation tables and figures |
| M9 — Registry/report service | Model, feature schema, threshold, dependencies and metrics | Package compatible versions and retain evidence | Versioned model package and report |

## 7. Data stores and interface contracts

| Store / interface | Contents | Rules |
|---|---|---|

| | | |
|---|---|---|
| D1 — Model package | Model weights/parameters, vectorizer, normalization version, class mapping, threshold/policy version, dependency manifest | Keep components version-matched; reject incompatible or missing artifacts. |
| D2 — Dataset and split manifest | Labelled URL records, provenance, registered-domain grouping and train/validation/test assignment | Offline only; fit transforms on training data only; test data never selects threshold. |
| D3 — Evaluation evidence | Metrics, confidence intervals, latency, seeds, configuration and limitations | Tie every report to dataset, split and model versions. |
| Request contract | Required field: url (string) | Reject malformed, empty or over-limit requests with a clear error. |
| Response contract | label; optional score and score semantics; model_version; policy_version; optional explanation/warning | Do not call a score a calibrated probability unless calibration was assessed. |

## 8. Requirement-to-design traceability

| Requirement | Mapped design element | Verification evidence |
|---|---|---|
| Accept URL input | M1 / runtime flowchart | Valid input accepted; invalid request returns controlled error. |
| Validate and normalize | M2 | Unit tests for malformed, Unicode, long and edge-case URLs. |
| Extract URL features | M4 | Feature schema, deterministic output and regression tests. |
| Optional DNS/domain data | Future feature-provider interface | Not active in current URL-only path; if added, test missing values, safety and latency. |
| Optional HTML/page data | Future isolated enrichment service | Not implemented; require safe-fetching design and legal/privacy review. |
| Preprocess and vectorize | M4 | Same fitted pipeline used in training and inference. |
| Classify URL | M5 + M6 | Held-out classification report and end-to-end test. |
| Return risk score | M5 + response serializer | Document score semantics and calibration status. |
| User-friendly result | M1 / response serializer | Clear verdict, explanation and error-state checks. |
| Explain prediction | M7 | Test supported explanations and measure latency/limitations. |
| Log/audit | M7 + D3 | Audit contains version metadata without unnecessary raw URL retention. |
| Controlled retraining | M3, M8, M9 | Candidate model must pass evaluation before promotion. |

## 9. Evaluation and quality controls

- Report accuracy, precision, recall, F1, confusion matrix, false-positive rate, and PR-AUC or another justified score-based metric.
- For the recall-first objective, report recall and false-positive rate at the selected threshold together; a recall guarantee is conditional on its calibration/test assumptions.
- Report both URL-level and registered-domain-level results where multiple URLs may belong to the same domain.
- Keep train, validation/calibration and test data separate; select thresholds on validation/calibration data only.
- Record random seeds, split manifests, data provenance, dependency versions, model version and configuration.
- Test URL-padding robustness as a simulated, non-adaptive transformation; do not claim adaptive-attack resistance.
- Because the supplied roadmap states that trustworthy timestamps are unavailable, do not claim temporal generalization or zero-day performance.
- Current explanation timing in the supplied roadmap is approximately 2 ms/URL for linear explanations and under 1 ms for CNN saliency; SHAP/LIME comparison and human evaluation remain future work.

## 10. Risks and future extensions

| Risk / extension | Design response |
|---|---|
| Dataset/source shortcuts | Audit URL construction and source features; use registered-domain-disjoint splits and, if possible, a second public dataset. |
| High false-positive cost at high recall | Make threshold trade-offs explicit; report recall with FPR and use validation-only threshold selection. |
| CNN/ensemble does not reliably beat TF-IDF | Keep the simpler baseline unless paired, domain-level results justify added complexity. |
| URL-padding evasion | Keep robustness transforms reproducible and report the clean-performance cost of augmentation. |

| | |
|---|---|
| Temporal drift | Requires trustworthy timestamped phishing first-seen data and dated legitimate snapshots; not supported by current data. |
| DNS/TLS/page fusion | Future extension only; collect safely, review legal/privacy constraints, and measure p50/p95 latency and recall gain at fixed FPR. |
| Model update failure | Use immutable model versions, compatibility checks, candidate evaluation and rollback. |

## 11. Conclusion

The detailed design provides the required flowcharts, UML class and sequence diagrams, and DFD Level 0 and Level 1. The aim/objective flowcharts connect the literature survey to gap identification, requirements and evaluation planning. The design remains consistent with the Task 2 architecture and explicitly separates currently supported URL-only capabilities from future work. This traceable structure supports implementation, testing, reproducibility and a defensible research report.